

ФСП Таланты

LLH_BY_KARMA · Документация демонстрационной сборки

Исходники: ветка main репозитория NedYouNet/LLH_BY_KARMA. Основа: backend 49303e2 и frontend 5272f36, дополненные исправлениями интеграции.

Независимое тестирование запланировано на 11 октября 2026 года. Этот документ описывает текущую реализацию и выполненные автоматические проверки, а не результат ещё не проведённой проверки.

1. Архитектура

Браузер открывает React-приложение. nginx в контейнере frontend пересылает /api/ в FastAPI backend:8000. При разработке ту же роль выполняет прокси Vite. Авторизация, роли, попытки тестов, доступ к контактам и решение о грейде остаются в бэкенде. Данные хранятся в PostgreSQL; схему обновляет Alembic.

ML работает в том же Python-приложении: app/ml/resume.py разбирает текст PDF через GigaChat при наличии ключа, app/ml/matching.py локально вычисляет пересечение токенов. app/testing_bank/bank.py содержит 60 шаблонов задач. Сервер на порту 8001 не входит в эту сборку.

testing_engine.py генерирует и оценивает задания; testing_service.py хранит попытки, таймеры, проверки и историю грейда. matching_engine.py считает оценку соответствия; matching_service.py применяет её к данным базы. ФСП и ATS имеют отдельные сервисы и маршруты API.

Контейнеры: frontend (nginx), backend (Python 3.12), db (PostgreSQL 16), mailpit (тестовая почта). Публичные локальные порты: 5173, 8000, 8025. PostgreSQL доступен только внутри сети Compose.

Отдельные прототипы веток feature/ml-core и Data_Scientist не разворачиваются параллельно. Их перенос в основной API требует адаптации контрактов, а не копирования собственного main.py поверх приложения.

2. Механика тестирования

Кандидат выбирает направление и целевой грейд. Пять направлений × четыре грейда дают 20 сочетаний. Для каждого есть три Python-задачи: easy, medium, hard. Intern соответствует уровню 0, Junior — 1, Middle — 2, Senior — 3. Направление и грейд проверяются схемами API.

Новый тест всегда состоит из трёх задач строго целевого грейда. План 5/3/2 не применяется к новым попыткам. Идентификатор шаблона содержит уровень и сложность; движок выбирает по одной задаче каждой сложности. Предпочтительные навыки вакансии могут влиять на выбор, когда для одной сложности есть несколько шаблонов. В текущем банке обычно доступен один.

Легенды подготовлены заранее. Template.make(rng) вызывается при старте теста. Некоторые функции используют случайные данные, некоторые пока возвращают фиксированные случаи. Скрытые тесты сохраняются в серверной попытке; публичный ответ содержит только пример текущего варианта. Наличие вариантов уменьшает прямое копирование ответов, но не гарантирует защиту от списывания.

Сложность	База	Бонус	Максимум
easy	15	5	20
medium	20	10	30
hard	35	15	50

Баллы = база × passed / total + бонус при passed = total. Например, сложная задача с 4 из 5 тестов даёт 28 баллов; с 5 из 5 — 50. Общий балл округляется один раз до одного знака, результат задачи — до двух. Старые сохранённые задания без новых весов оцениваются прежней взвешенной формулой: исторические записи не переписываются автоматически.

POST /api/testing/attempts/{id}/check принимает item_id и code. На каждую задачу предусмотрено 10 проверок; счётчик сохраняется сервером и не сбрасывается перезагрузкой страницы. Проверка не завершает попытку и не присваивает грейд. Окончательная сдача оценивает последний отправленный код.

Грейд присваивает существующая логика бэкенда, а не LLM. Проходной порог — 70; при результате от 90 возможна рекомендация более высокого грейда. Повтор теста и изменение грейда ограничиваются настройками cooldown. Понижение подтверждённого грейда не выполняется.

Проверяется только Python. Функция должна соответствовать контракту задачи и возвращать значение; чтение stdin и печать результата не заменяют return. AST-фильтры и ограниченный subprocess отсекают часть опасных конструкций, ограничивают время и память. Это не промышленная песочница и не доказательство асимптотической сложности алгоритма.

3. Подбор и категоризация

Категория — направление и грейд. Подтверждённая категория берётся из завершённого теста; без теста используется заявленная категория с пометкой и понижающим множителем. Работодатель фильтрует выдачу и получает балл соответствия, причины и вклад факторов. Избранное хранится отдельно от оценки соответствия.

Текущая формула находится в backend/app/services/matching_engine.py; настройки — в app/core/config.py. Веса по умолчанию: тест 0.35, ФСП 0.15, навыки 0.35, текст 0.05, активность 0.10. Компоненты без применимого сигнала исключаются, оставшиеся веса нормируются. Неподтверждённый грейд получает множитель 0.6; соседний с целевым — 0.7; отличие на две и более ступени — 0.5.

При app/ml/matching.py: ENABLED=True текстовый сигнал — доля токенов запроса, найденных в тексте кандидата. Это локальный расчёт без LLM и без сетевого запроса на каждого кандидата. TF-IDF остаётся альтернативной веткой движка. breakdown, reasons, matched_skills и отсутствующие навыки объясняют расчёт интерфейсу.

Веса из автономного прототипа Data_Scientist не перенесены автоматически: его формула и API отличаются. Реальные профили берутся из базы основного приложения, а не из CSV. Качество ранжирования на размеченной выборке независимого оценщика в этой сборке ещё не подтверждено.

4. Проверки текущей сборки

Проверки выполнялись при подготовке сборки 10 октября 2026 года. Независимый тестировщик ещё не проводил итоговую проверку; она запланирована на 11 октября. Это отдельный этап, результаты которого следует добавить сюда после выполнения.

Проверка	Результат
<code>python -m pytest -q</code> на SQLite	169 тестов прошли; одно предупреждение устаревающего API зависимости Starlette/AnyIO
Покрытие банка	60 шаблонов, 5 направлений, 4 грейда, 3 сложности
Эталонные алгоритмы	20 сочетаний направления/грейда: эталонные алгоритмы прошли все тесты, 100/100
Сборка фронтенда	<code>npm ci</code> , <code>npm run lint</code> и <code>npm run build</code> прошли; Monaco включён в локальную сборку
Docker Compose и PostgreSQL на машине сборки	Не запускались: Docker здесь недоступен
Реальный GigaChat и настоящий FSP ID	Не подтверждены этим прогоном; тесты не используют реальные ключи
Независимые сценарии UI и реальная ATS	Предстоят

Автотесты проверяют API и права доступа, тестирование, баллы, сохранение проверок, контакты, ФСП, ATS и контракты ML. Проверка эталонных решений выполняет реальные Python-функции на тестах банка. Она подтверждает согласованность банка с выбранными эталонами, а не полноту всех граничных случаев.

Воспроизведение

Из backend/ с Python 3.12:

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
python -m scripts.verify_hybrid_testing
```

GitHub Actions дополнительно запускает тесты на PostgreSQL и цикл миграций. Статус CI следует проверить у опубликованного коммита; локальный SQLite-прогон не заменяет PostgreSQL-прогон.

`scripts.evaluate` моделирует кандидатов и синтетические пары для исследовательской оценки. Его результаты не равны проверке реальных кандидатов или независимой оценке качества ML. Старые цифры про тысячи вариантов, точность, утечки и нагрузку не переносятся в отчёт без актуального воспроизводимого прогона.

Перечень оставшихся сценариев и форма регистрации дефекта: QA_CHECKLIST.md. Дефект должен содержать роль, шаги, ожидаемый и фактический результат, HTTP-ответ и `health.build`.

Сборщик предупреждает о размере лениво загружаемого Monaco-чанка (~2.7 MB до gzip, ~699 kB gzip). Это локальная зависимость редактора; остальной интерфейс загружается отдельным меньшим чанком. Предупреждение не является ошибкой сборки.

5. Интеграция ФСП

backend/app/services/fsp_service.py и связанные сервисы/маршруты отвечают за привязку ФСП, достижения, разряд и вход FSP ID. Справочные результаты читаются из backend/data/fsp_results.xlsx; импорт и создание шаблона — scripts/fsp_data.py. При отсутствии записи применяется демонстрационный реестр, если включён fallback.

В локальном Compose FSP_ID_MODE=mock: вход имитирует провайдера для демонстрации. Настоящая OIDC-интеграция требует issuer, client ID, секрета и регистрации приложения у провайдера. Работу с настоящим провайдером команда должна проверить отдельно; mock не является такой проверкой.

Фронтенд получает параметры через основной API /api/auth/fsp-id/config, показывает сведения ФСП и разряд в профиле/карточке. Секрет клиента остаётся на сервере. История ФСП — один из факторов ранжирования, а не замена результата теста.

Свежие адреса и схемы находятся в Swagger. Таблица результатов в репозитории — шаблон/демо-источник; реальные персональные записи перед публикацией должны быть отделены от публичных материалов.

6. API

Единственный API для интерфейса — /api/... основного FastAPI. Swagger: http://localhost:8000/docs. JSON-схема при запуске: http://localhost:8000/openapi.json. В docs/openapi.json сохранён снимок схемы этой сборки.

Авторизация: POST /api/auth/login; далее Authorization: Bearer <access_token>. Роль проверяет бэкэнд. Основные группы маршрутов: auth, candidate, employer, testing, invitations, vacancies, applications, short tasks, FSP и ATS. Точные тела и ответы проверяйте по схеме, а не по API автономных прототипов.

Новое в тестировании: POST /api/testing/attempts/{id}/check с {"item_id": "q1", "code": "def solve(...): ..."}.

Ответ содержит passed, total, checks_used, checks_remaining, check_limit и безопасные сообщения. Публичные задания содержат difficulty, base_weight, bonus_weight, max_score и счётчик проверок. Скрытые тесты и правильные ответы не возвращаются.

Загрузка PDF: POST /api/candidate/resume/parse, multipart-файл. Для GigaChat следует предусмотреть ожидание до 20 секунд и запас по таймауту интерфейса. source указывает ml/fallback. Все обращения идут через бэкэнд.

GET /api/health сообщает статус, окружение, версию и build (отпечаток кода и миграция). Отпечаток меняется после изменения файлов; не сверяйте его со старым фиксированным значением из переписки.

Таблица всех маршрутов автоматически обновляется командой python -m scripts.export_api из backend/. Обзор этого раздела при экспорте не перезаписывается.

7. Зависимости

Python 3.12; полный фиксированный список — backend/requirements.txt, зависимости проверок — requirements-dev.txt. Основные: FastAPI 0.115.12, Uvicorn 0.34.2, SQLAlchemy 2.0.41, Alembic 1.16.1, pycorg 3.2.9, Pydantic 2.11.5, httpx 0.28.1, ruypdf 5.6.0, ReportLab 4.4.1, openpyxl 3.1.5, cryptography 45.0.5, GigaChat SDK 0.2.3. pytest 8.3.5 — только для проверок.

Frontend: Node.js 22 в Docker, React 19, Vite 8, Tailwind CSS 4, Monaco Editor через @monaco-editor/react. Диапазоны — в frontend/package.json, точные версии — в frontend/package-lock.json; установка через

`npm ci`.

Контейнеры: PostgreSQL 16 Alpine, nginx 1.27 Alpine, Mailpit. Mailpit использует тег latest и поэтому не фиксирует точный образ; для воспроизводимого промышленного развёртывания нужно закрепить digest. GPU и отдельный ML-сервер для текущей сборки не требуются.

Tesseract, PyMuPDF и отдельные зависимости CSV-прототипа участника 4 в основной сборке не установлены: OCR оттуда не подключён.

8. Запуск, обновление и остановка

Чистая установка

Включите Docker Desktop. Скачайте ветку main, откройте PowerShell в папке с `docker-compose.yml`. Выполните `docker compose up -d --build`, затем `docker compose exec backend python -m scripts.seed`. Откройте <http://localhost:5173>. Миграции применяются автоматически. Для тестовой почты откройте <http://localhost:8025>.

Если уже работали из другой папки

Сначала в старой папке выполните `docker compose stop`. Не запускайте одновременно старую и новую сборку на портах 8000/5173. Для самостоятельной чистой проверки в новой папке можно задать отдельное имя: `docker compose -p llh_main up -d --build`; все последующие команды также выполняйте с `-p llh_main`. Это создаст отдельный том и не перенесёт старую базу. Чтобы сохранить старую базу, оставайтесь с прежним именем Compose-проекта и сначала сделайте резервную копию данных.

Не копируйте `.env` и локальную базу в публичный Git. Старый `docker-compose.integration.yml` для этой сборки не нужен: текущие настройки ML уже находятся в основном `compose.backend/.env` для локального Python и корневой `.env` для Compose имеют разные области применения.

Обновление после публикации

Сохраните свои изменения коммитом в рабочей ветке. Для отдельной чистой копии main: `git pull --ff-only`, затем `docker compose up -d --build`. Проверяйте `docker compose ps`, `/api/health` и `docker compose logs backend --tail 100`. Не считайте один только номер версии доказательством нового кода: смотрите поле build и исходный коммит.

Seed без reset предназначен для первой инициализации. `docker compose exec backend python -m scripts.seed --reset` удаляет тестовые аккаунты, вакансии, ответы и попытки, после чего создаёт демо заново. Используйте только если эти записи не нужны. После reset выйдите и войдите заново. `docker compose down -v` также уничтожает том базы и не нужен для обычного обновления.

Разработка интерфейса

Из корня: `docker compose stop frontend`. Затем `cd frontend`, `npm ci`, `npm run dev`. Vite пересылает `/api` на `localhost:8000`. Если бэкэнд на другом компьютере, задайте `BACKEND_URL=http://<IP>:8000` в `frontend/.env.local` и настройте доступ к API; текущий compose привязан к localhost. Сборка: `npm run build`. Проверка кода: `npm run lint`.

Остановка

`Ctrl+C` останавливает foreground-процесс, например `npm run dev`. Docker, запущенный с `-d`, работает в фоне: остановите его командой `docker compose stop`. Для продолжения — `docker compose start`. После изменения исходников — `docker compose up -d --build`.

Для сервера

Этот compose — локальная dev-сборка. Перед внешним развёртыванием нужны HTTPS, prod-настройки и собственные секреты, подтверждённая интеграция ФСП/ATS, изолированная среда исполнения кода, резервное копирование и проверка PostgreSQL/нагрузки. Здесь эти этапы не объявлены выполненными.

9. Обработка данных и ограничения безопасности

Бэкенд проверяет JWT и роли, маскирует контакты кандидата до разрешения работодателю, хранит согласия и поддерживает отзыв доступа. Поля персональных данных шифруются через AES-GCM; dev-ключ предназначен для локального стенда, а для prod нужны собственные ключи и процесс их хранения/ротации.

Перед отправкой текста резюме в GigaChat удаляются распознанные email, телефоны и Telegram. Регулярные выражения уменьшают раскрытие контактов, но не гарантируют удаление всех персональных данных из произвольного текста. Ключ GigaChat не отправляется в браузер и не хранится в репозитории.

Скрытые тесты остаются на сервере. Проверка кода использует AST-ограничения, таймаут и ограничения ресурсов. Исполнение произвольного Python в subprocess не является достаточной промышленной изоляцией. Перед публичным приёмом пользовательского кода нужна отдельная безопасная среда исполнения.

Наличие этих механизмов не является юридическим заключением о соответствии 152-ФЗ. Для реального запуска отдельно определяются оператор, основания обработки, сроки хранения, политика, доступы и договоры с внешними сервисами. Настройки TLS внешнего ML-клиента также требуют проверки перед prod.

10. ATS и разрешение контактов

В dev встроен тестовый приёмник ATS. Под работодателем сохраните `webhook_url = http://localhost:8000/api/mock-ats/webhook` через `PUT /api/employer/integrations/ats`. Проверка: `POST /api/employer/integrations/ats/test`, ожидается `ok: true`. `GET /api/mock-ats/events` возвращает последние события; смотрите `signature_valid`.

Событие принятия приглашения — `invitation.accepted`. Это локальная демонстрация доставки, а не настройка реальной внешней ATS. В prod mock-приёмник выключен; для настоящей интеграции нужны адрес приёмника, проверка подписи и обработка повторной доставки на стороне ATS.

Кандидат управляет разрешением контактов. Работодатель видит данные только при действующем разрешении. Независимому тестировщику нужно проверить обе роли, отзыв разрешения и отсутствие контактов в закрытых API-ответах, а не только текст плашки на экране.

11. ML в итоговой сборке

Встроенные модули находятся в `backend/app/ml/`; банк задач — в `backend/app/testing_bank/bank.py`. Отдельный FastAPI на 8001 и файлы автономного `feature/ml-core` не требуются.

Тестирование использует подготовленные тексты и серверные функции генерации данных. GigaChat во время прохождения теста не вызывается. Текущий банк: 60 размеченных Python-шаблонов для пяти направлений и четырёх грейдов. Для каждого сочетания есть `easy`, `mid`, `hard`. Общий алгоритмический каркас повторяется между направлениями; это ещё не полноценное покрытие профессиональных навыков каждого направления. Часть генераторов имеет фиксированные данные

— их дальнейшая параметризация остаётся задачей ML-разработчика.

Резюме: `rupdf` извлекает текст, сервер маскирует контакты, `app/ml/resume.py` при наличии ключа вызывает GigaChat (модель по умолчанию GigaChat-Pro). При отсутствии ключа/ошибке работает встроенный парсер. Время ожидания модели ограничено. OCR сканов не интегрирован. Реальное качество разбора резюме нужно измерить на согласованной выборке.

Подбор: `app/ml/matching.py` считает локальное покрытие токенов запроса. Это не LLM-объяснение и не эмбединг-модель. Итоговую оценку, фильтры и понятные причины рассчитывает `matching_engine.py`. Маршрут объяснения, если оставлен в API, не является обязательным этапом выдачи.

Эталонные решения и проверки банка: `scripts/verify_hybrid_testing.py`, `tests/test_contest_generation.py`, `tests/test_all_contests.py`, `tests/test_contest_scoring.py`. Успех этих проверок не доказывает качество профессионального грейдинга, отсутствие списывания или полноту скрытых тестов.

Передача работы и следующая проверка

Отдельные текстовые файлы: `docs/handoff/backend.txt`, `ml.txt`, `participant_4.txt`. План независимого тестирования: `docs/QA_CHECKLIST.md`. Схема API и таблица маршрутов: `docs/openapi.json` и `docs/api_routes.md`. Точные версии frontend — `package-lock.json`; backend — `requirements.txt`.

Приоритеты независимой проверки

- Чистый запуск Docker, миграции PostgreSQL и создание демо-данных без удаления существующих записей.
- Все 20 сочетаний направления и грейда: нужные задачи, 20/30/50 баллов, частичные решения и бонус только за все тесты.
- Предварительные проверки: сохранение счётчика, отказ после десятой, параллельные нажатия, чужие и завершённые попытки.
- Согласованность итогов: результат теста, профиль кандидата, история грейда и карточка работодателя; таймер и `cooldown`.
- Две роли: вакансии, отклики, короткие задания, приглашения, согласие и отзыв доступа к фактическим контактам.
- ML-резюме, ФСП и ATS: отдельно пометать `mock/fallback` и реальные внешние интеграции. Проверить обе темы и мобильный интерфейс.

Для дефекта укажите роль, шаги, ожидаемое и фактическое поведение, HTTP-ответ без секретов и отпечаток сборки. После проверки добавьте фактические результаты в `docs/04_validation.md`; текущий статус остаётся «не выполнено».